

Simple Recurrent Networks (SRNs)

Computational Cognitive Neuroscience – Midterm mini-project

Total points: 10 + optional bonus (2 points)

Instructions

- Be sure to follow the instructions for downloading the emergent software from Piazza before beginning the midterm.
 - Collaboration policy: The midterm is to be done **individually**, you are not allowed to discuss any aspect of the midterm (software, conceptual or otherwise) with your classmates. Any form of collaboration on the midterm will constitute violation of the University academic code.
 - At the beginning of every answer, specify the input sequence you are using as well as specific model parameters (if any) that you are using for simulation. You will lose points if your answer is unclear about the model inputs or specific parameter settings.
 - If you have any clarification questions or concerns, please write to the course instructor and TAs.
-

Introduction

This midterm mini-project has the dual purpose of exposing you to the emergent code -- you will get instructions to build and train a network from scratch -- and exploring the power of simple recurrent networks (SRNs) to learn sequences. We'll see how a network can learn to predict the next item in a sequence depending on its current input (e.g. predicting the next word in a sentence based on the current and previous words). As a reminder, this topic is not currently covered as such in the online text, but is covered in the lecture, so you may want to refer to the slides on "Temporal learning and representation".

Why do we need a special type of network for this? After all, we have seen that networks can learn to associate specific inputs with specific outputs using error-driven and self-organized learning. So if I'm in the middle of an input sequence that I've seen before, then shouldn't a trained network be able to predict the next item easily? Well, it all depends on how complicated the sequence is. ABCDEF is easy -- there's never any ambiguity about what the next letter should be. ABCBDBEBF is not, on the other hand. It's not clear what item should come after a B, unless you look further back in the sequence for some disambiguating context. That's what SRNs use a **context layer** for.

Before getting started on building and modifying our network, you should use **core run** (run from the terminal in the directory with `srn.go`) to build the `srn.go` file. The GUI window should automatically open once it is successfully compiled. You should see that it consists of two layers and that it doesn't do anything. We're going to change that! To get oriented, let's first take a brief tour of the `go` code. It will be helpful to keep Emergent's basic hierarchy in mind. Recall that:

1. The **network** is composed of **layers**

2. **Layers** are groups of **units** (neurons), which are organized together so that **projections** (synaptic connections) can be specified between whole groups which roughly share some function (e.g. representing conjunctions of lines).
3. When training the network, we *force* **Input** and **Output** layers to have the activity that we want the network to reproduce. Technically, layers we clamp on are actually called **Target** layers, but those are usually outputs.
4. **Hidden** layers are layers that we generally consider inaccessible to the outside world, where "the outside world" could be other brain areas, an experimenter, etc.
5. Time in emergent is organized into **cycles**, which make up **trials**. A trial is the time period during which each particular input or input/output pair is given. Groups of trials form **epochs**. These are useful because we may want to test a network's performance on some task after it's seen a specified number of training examples, such as after one round of seeing all the different input/output pairs.
6. **Runs** are groups of epochs. If we train a network and observe it's performance over time, this may or may not be representative of how most networks would behave. Thus we can run a set of epochs, record performance, initialize the network again, and repeat to get a better idea of the general case.

Tour of the project code

Please open the `srn.go` file in a text editor and search for "Landmark 1". You'll make two basic stops, then look around a bit. After you do that, come back here.

Section 1. Simple sequences without context layer

Next we'll create a basic network with a simple sequential input, and see whether the network can learn this sequence. To do so, find "Section 1a" in the project code, then follow the instructions from there.

Once you've completed the subsections 1a-1b, which set up the network and create input, you should step through some trials (with `init`, then `step trial`). Verify that you understand the relationship between the data table you created and which units are on. You can also observe this by clicking the Zeroth button on the side panel. This should display the contents of the training file you created.

Question 1: Train the network. Does it learn to produce the correct outputs for the given inputs?

Question 2: What do you think would happen during training if you had a sequence like ABCBA? Why?

Section 2. First order dependencies

Now we'll try using longer sequences with first order dependencies (e.g. ABCBA) and see if this

standard hidden-layer model can solve these tasks. One thing that's good to do, so that you can compare variations of a model is to make multiple input data tables, rather than repeatedly editing the same one. To use a first order sequence, just select First button under Patterns on the left-hand side panel. You can verify that it loads a first order sequence by clicking on First on the side panel. You should see an input sequence such as ABCBA, where at least one input predicts different outputs depending on what came before it. The names of these might be A, BC, C, BA, and A, so that you can easily distinguish which "B" trial you're in in the sequence. It is *very important* that the *input* for the trial named BA be the *same* as the input for the trial named BC. This is what will force the network to hold onto context.

Once you have inspected the input sequence, try training the network to learn the first order sequence. Examine the Train Epoch Plot to see if the network was able to learn the sequence.

Question 3: Was it able to learn this sequence? Why or why not? Explain any discrepancy between what you expected and what you observed.

Section 3. Adding a context layer

As you should have observed, our current model has trouble learning the higher order sequence (if it did learn your sequence, you must have done something wrong setting up the Network, or the sequence). As you proceed through this section we will explore how adding a context layer allows the network to discern which of several outputs is required given a single input, based on the context. We will learn that second and third order dependencies are harder for the network to learn but that adjusting the rate at which the network takes activity from the hidden layer can help.

Go back to Section 1a in the code, and create another layer just like the hidden layer, but named "Context". Write another `net.ConnectLayers(...)` statement that's just like the one connecting the input to the hidden layer, but which goes from the hidden context layer you just created to the hidden layer. Next, in the same `ConfigNet()` function, you will need to adjust the GUI so that the context layer is displayed neatly. Search for "Section 3: Context Layer GUI Placement" in `ConfigNet()` function to locate this part of the code.

Once you've done this, insert the following code at the end of the `ApplyInputs()` function. It's somewhat specific to this project. Can you see what it does just by reading it?

```
context_in := ss.Net.LayerByName("Hidden")
context_out := ss.Net.LayerByName("Context")

context_in.UnitValues(&ss.TmpVals1, "ActP", -1)
context_out.UnitValues(&ss.TmpVals2, "ActP", -1)
for i,_ := range ss.TmpVals1 {
    ss.TmpVals1[i] = ss.TmpVals1[i] * 1.0 + ss.TmpVals2[i] * 0.0
}
context_out.ApplyExt1D32(ss.TmpVals1)
```

Also (**important**), you'll need to comment out (i.e., insert //) the line at the beginning of the `ApplyInputs()` function which reads `net.InitActs()` - we don't want to reset the activations for the SRN!

Re-build the network (i.e., do core run) and verify that now there's an additional hidden layer. Then train the network again. You should see activity in the new context layer, and it should be a copy of the hidden layer (of the last trial)! If you look at sending and receiving weights, these should accord with your modifications to the code.

It is a good idea to get a sense of the variability of learning, because some networks will learn faster than others due to randomness in the initial weights. You should allow the first network to run for a few hundred epochs and then also train a series of networks rather than just a single one, by modifying the number `NRuns` in the Config on the left-hand sidebar. You can then see how much variability there is by looking at the overlaid plots in the Train Run Plot tab or by inspecting the various logs in the control panel. For example, you can see summary information for the different runs by looking at the Train&Run log table.

Question 4: Is the network able to learn the sequence this time? Explain how the context layer allows learning of sequences with higher orders.

Section 4. Second-order dependencies.

The example sequence A B C B A only has first order dependencies because you would only have to remember the very last item before the repeated item to answer correctly. A sequence with second order dependencies could look like this: A B C D B C E F. To predict E when given C it is not sufficient to keep remember the previous B trial, because that is the same as earlier in the sequence; You need to know that the BC sequence was preceded by D, two trials back. Likewise, a third order dependency would be something like A B C D E B C D F.

To use second or other higher order sequences, edit the `custom.tsv` file and select Custom button on the side panel. Create a second order sequence. See whether the current network can learn this task (i.e. whether it can get to zero errors, and also how long it takes to do so). Now experiment with third and fourth order sequences. Feel free to try other higher order sequences as well.

If you are having trouble getting your network to learn a higher order contingency, besides trying to manipulate parameters you should make sure that you didn't just get unlucky with the initial weights. (Same thing if your network seems to have learned higher order contingencies very easily, you might have just gotten lucky!). You should train a sequence of networks as specified above (before Question 4).

Question 5: Is the network able to learn the sequence this time? Why or why not? And if so, in how many epochs?

In order for the network to do better at remembering second and third order dependencies we can change the rate at which the context layer takes information from the hidden layer. Although we said that it "copies" the previous hidden layer activity, this is actually a 'special case' of the SRN, and this function can easily be altered so that the context layer activities reflect part of the previous hidden layer activity but also part of the previous context activity itself. This makes the network more likely

to hold onto information from the past.

This is controlled by the parameters named `FmHid` and `FmPrv`. The `FmHid` parameter controls the percentage of the context layer activity that will be taken directly from the hidden layer. The `FmPrv` parameter controls the percentage of the context layer activity that will be taken from the previous context activity (on the last trial, but again remember that this last trial will in turn be incorporating activity from the trial before that, and so on).

Go into the `srn.go` file, and find the `FmHid` and `FmPrv` parameters. They're in the definition of the `Sim` structure, and the `New()` method of `Sim`. Why? Uncomment the parameters and their default settings. Go to the code snippet you inserted earlier that starts with `context_in` and replace the `1.0` and `0.0` with `ss.FmHid` and `ss.FmPrv` respectively. Rebuild the network and you should see them appear in the control panel.

Question 6: What values of `FmHid` and `FmPrv` allow your network to learn the second order dependency of your previous sequence from Section 2? If it learned already, what values allow it to learn with reduced training time (number of epochs)? Optional: if you tried a third order dependency, what values are needed to more reliably learn this or to optimize the speed at which it learns?

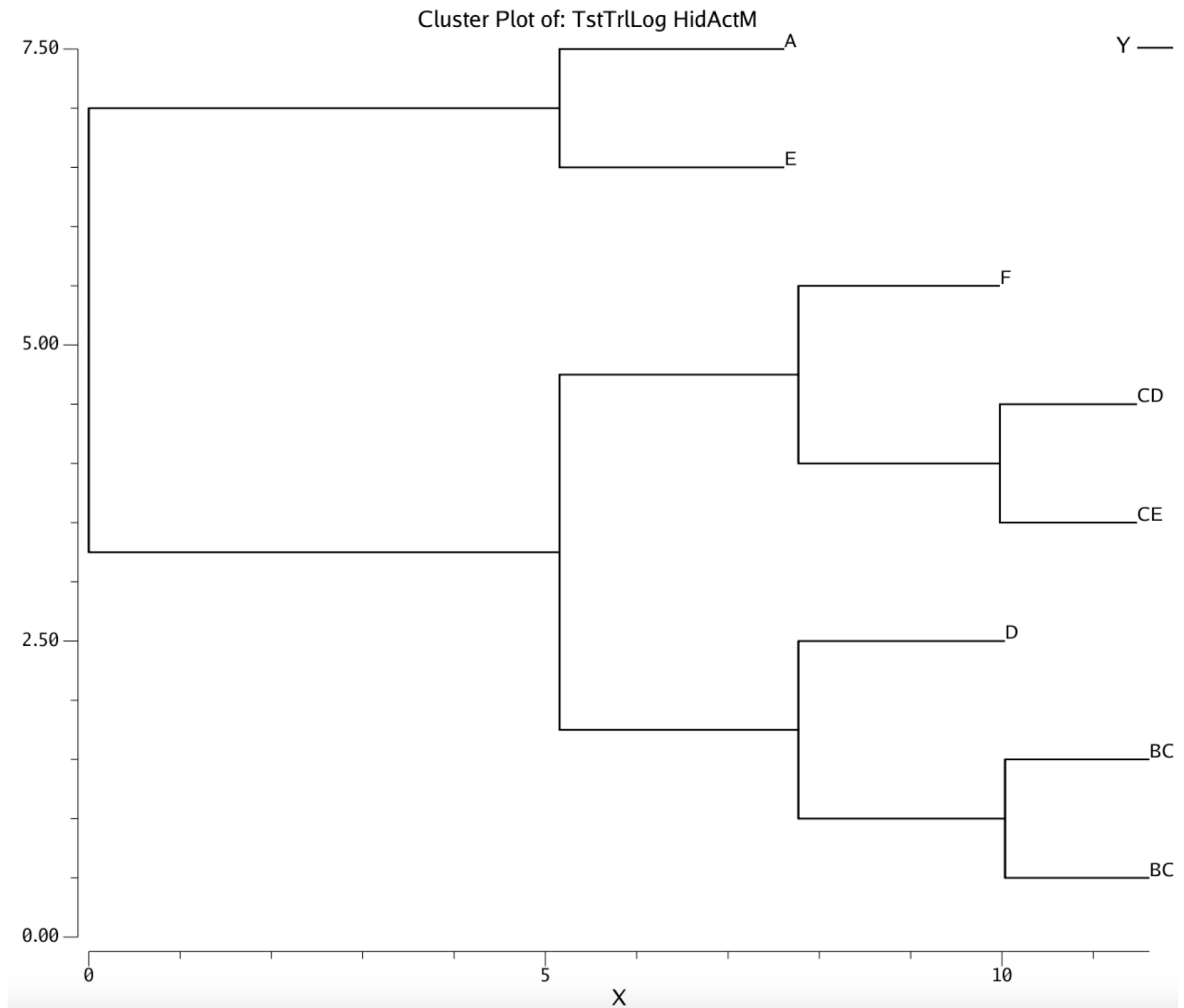
Question 7: Why do these parameters change the network's ability to respond appropriately given different sequential dependencies?

Now we will take a look at what the Hidden layer units are representing. We will examine the cluster plot from the hidden layer activations. The code for generating the cluster plot is a little complicated, which is why you're not creating it yourself, but you could refer to the HW simulations for the function definition, read it over and give it some thought, as usual. The `family_trees` simulation from Chapter 4 assignment should have a cluster plot function defined.

For this section, instead of implementing and generating a cluster plot by yourself, you will be provided with the input sequence and the corresponding hidden layer cluster plot. You will need to interpret what the cluster plot suggests about the hidden layer's representations.

Input sequence: A B C D B C E F

Cluster plot:



Question 8: Which patterns are clustered together and why? How strongly similar are the clustered patterns?

Remember: The total horizontal distance that you have to traverse from two distinct pattern-names to their common branching point in the cluster plot is equal to the similarity of these patterns. If you're not sure about the cluster plot, think about it while actually stepping through and watching the hidden layer!

Question 9: What is the greatest order of dependency that you can still train the network on? This is your chance to show off! What values of FmHid and FmPrv did you need to use, and why?

(Optional - 2 bonus points) Feel free to make any other customizations you think may help as well, but be sure to document them here.

Question 10: Take a moment to consider what SRNs do and how they function. What are some examples of cognitive process or phenomena that they might capture behaviorally?